

Python Variables

Technical Notes

Rodrigo Kang

Table of contents

1	Programming Foundations with Python	3
1.1	Essential Concepts of Computation	3
1.2	Python Scripts and Interactive Computing	5
1.2.1	The Jupyter Ecosystem	6
1.2.2	IPython and Jupyter Magic Commands	6
1.3	Python Modules: Reusable Code and Namespaces	7
1.3.1	Importing Modules	8
1.3.2	Inspecting Modules and Namespaces	9
1.3.3	Module Loading and Reloading	12
1.4	Python Execution from the Command Line	12
1.5	Environment and Package Management with Conda	14
1.5.1	Reference Resources	15
1.6	The Python Ecosystem	16
1.7	Representing Information in Python	16
2	Working with Python Variables	17
2.1	Multiple Assignment	18
2.2	Variable Naming	19
2.3	Python Keywords	20
2.4	Reassignment and Operations with Variables	21
2.5	Comments	23
3	Problems	23
3.1	Problem 1 — From Algorithm to Execution	24
3.2	Problem 2 — Compiled or Interpreted?	24
3.2.1	Questions	24
3.3	Problem 3 — Programming Paradigms	24
3.3.1	Questions	25
3.4	Problem 4 — Script or Interactive Environment?	25
3.4.1	Questions	25
3.5	Problem 5 — Magic Commands and Shell Commands	25
3.5.1	Questions	26

3.6	Problem 6 — Modules and Namespaces	26
3.6.1	Questions	26
3.7	Problem 7 — Import Styles	26
3.7.1	Questions	27
3.8	Problem 8 — Inspecting a Module	27
3.8.1	Questions	27
3.9	Problem 9 — Module Caching	27
3.9.1	Questions	28
3.10	Problem 10 — Python Environments	28
3.10.1	Questions	28
3.11	Problem 11 — Conda Commands	28
3.11.1	Follow-up Questions	29
3.12	Problem 12 — Python Ecosystem	29
3.12.1	Questions	29
3.13	Problem 13 — Objects and Types	30
3.13.1	Questions	30
3.14	Problem 14 — Scalar Types and Data Structures	30
3.14.1	Questions	30
3.15	Problem 15 — What Is a Variable in Python?	31
3.15.1	Questions	31
3.16	Problem 16 — Assignment Is Not Equality	31
3.16.1	Questions	31
3.17	Problem 17 — Names and Objects	32
3.17.1	Questions	32
3.18	Problem 18 — Multiple Assignment	32
3.18.1	Questions	32
3.19	Problem 19 — Variable Swapping	32
3.19.1	Questions	33
3.20	Problem 20 — Valid and Invalid Identifiers	33
3.20.1	Follow-up Questions	33
3.21	Problem 21 — Naming Conventions	33
3.21.1	Questions	34
3.22	Problem 22 — Python Keywords	34
3.22.1	Questions	34
3.23	Problem 23 — Reassignment and Dynamic Typing	34
3.23.1	Questions	35
3.24	Problem 24 — Augmented Assignment	35
3.24.1	Questions	35
3.25	Problem 25 — Predict the Result of Augmented Assignment	35
3.25.1	Follow-up Question	36
3.26	Problem 26 — Comments and Code Quality	36
3.26.1	Questions	36
3.27	Problem 27 — Namespace Collision	36
3.27.1	Questions	37
3.28	Problem 28 — Reproducibility in Interactive Computing	37
3.28.1	Questions	37

3.29 Problem 29 — Environment Reproducibility Scenario	37
3.29.1 Questions	38
3.30 Problem 30 — Integrated Technical Scenario	38
3.30.1 Questions	38

1 Programming Foundations with Python

Python is one of the most widely used programming languages in Data Science. Its adoption is largely driven by a combination of readability, expressive power, and a mature open-source ecosystem. Libraries such as NumPy, pandas, Matplotlib, scikit-learn, TensorFlow, and PyTorch provide a broad scientific computing environment capable of supporting tasks ranging from data ingestion and exploration to modelling, deployment, and automation.

Beyond its practical utility, Python encourages a particular approach to programming: clarity over unnecessary complexity, readability over cleverness, and explicitness over ambiguity. Many of these principles are captured in *The Zen of Python* and are reflected in the design and conventions of the language.

For Data Science, proficiency in Python involves more than familiarity with syntax or individual libraries. It requires an understanding of how information is represented, how algorithms transform that information, how programs are structured, and how computational instructions are ultimately executed.

1.1 Essential Concepts of Computation

At its core, **computation** can be understood as a structured transformation of information. A computational process typically receives some form of input, applies a defined procedure or algorithm, and produces an output.

A computational system can be considered in terms of two closely related components: **hardware** and **software**. Hardware comprises the physical components of a computer system, including processors, memory, storage, and input/output devices. Software provides the instructions and abstractions that determine how those physical resources are used to perform computational tasks.

An **operating system** provides the fundamental software layer between applications and the underlying hardware. It manages computational resources such as processors, memory, storage, files, and input/output devices, while providing services through which programs can be executed. Common operating systems include Linux, Windows, and macOS.

An **algorithm** is a finite and well-defined sequence of instructions designed to solve a problem or perform a task. Algorithms are abstract: they are independent of the particular programming

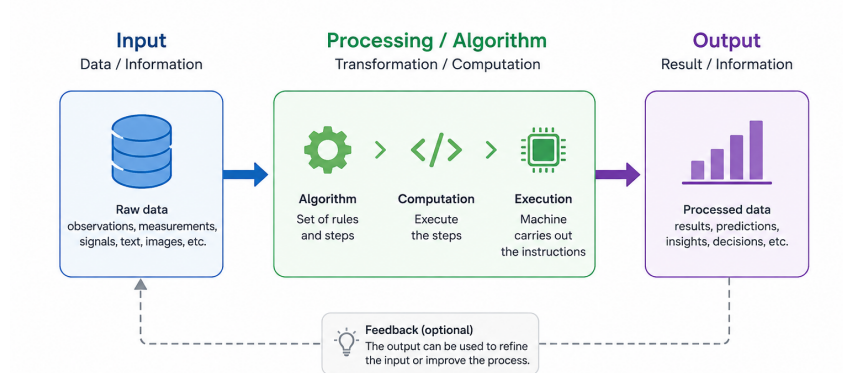


Figure 1: A basic computational process in which input data are transformed into an output through a defined algorithm or processing procedure.

language used to implement them. A programming language provides a formal mechanism for expressing those algorithms in a form that can ultimately be executed by a computer.

Programming languages are commonly described according to their level of abstraction. **Low-level languages** provide relatively direct control over machine operations, whereas **high-level languages** provide abstractions that allow programs to be expressed in forms closer to human reasoning and problem formulation.

The distinction between **compiled** and **interpreted** languages is useful but should not be treated as absolute. Source code may pass through several stages of compilation and interpretation before execution. In the standard CPython implementation, for example, Python source code is compiled into an intermediate representation known as **bytecode**, which is then executed by the Python virtual machine.

Programming languages can also support different **programming paradigms**, which provide alternative ways of structuring computational logic. Common paradigms include procedural, object-oriented, functional, and declarative programming. Python is a **multi-paradigm language**, supporting procedural, object-oriented, and functional programming styles.

A **program** is an organized collection of instructions that implements one or more algorithms. Python source code is commonly stored in plain-text files with the .py extension. Programs can range from small scripts designed to perform a specific task to large software systems composed of multiple packages and modules.

Python itself is a high-level, general-purpose programming language designed with a strong emphasis on readability and simplicity. Its relatively concise syntax, extensive standard library,

large open-source ecosystem, and support for multiple programming paradigms have made it particularly important in scientific computing, Data Science, Machine Learning, automation, and software development.

The design philosophy of Python is summarized in *The Zen of Python*, which can be displayed directly from the interpreter:

```
1 # Zen of Python
2 import this
```

The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

1.2 Python Scripts and Interactive Computing

Python scripts are plain-text files containing Python source code, typically identified by the .py extension. They constitute one of the fundamental ways of organizing and executing Python programs.

A script can be executed by passing its filename to the Python interpreter:

```
1 python script.py
```

Depending on the operating system and Python installation, the Python 3 interpreter may instead be exposed explicitly as python3:

```
1 python3 script.py
```

Python development can also be supported by **Integrated Development Environments (IDEs)** and code editors, which provide features such as syntax highlighting, code completion, debugging, source control integration, and project management.

For exploratory analysis and scientific computing, Python is also commonly used through **interactive computing environments**, particularly Jupyter.

1.2.1 The Jupyter Ecosystem

The **Jupyter Project** provides an open-source environment for interactive computing, combining executable code, documentation, mathematical notation, and visualization within the same computational document.

Originating from the IPython project in 2014, Jupyter evolved into a multi-language execution environment. Its name reflects three of the languages associated with its origins: **Julia, Python, and R**.

The Jupyter ecosystem includes several related tools:

- **Jupyter Notebook** — an interactive environment for creating computational documents that combine code, text, mathematical expressions, and visualizations.
- **JupyterLab** — a web-based development environment that extends the notebook experience with file navigation, terminals, text editors, and other integrated tools.
- **JupyterHub** — a multi-user platform for deploying and managing Jupyter environments.

1.2.2 IPython and Jupyter Magic Commands

IPython and Jupyter provide **magic commands** for common interactive operations. Line magics begin with %, while cell magics begin with %%.

The available magic commands can be inspected with:

```
1 %lsmagic
```

Some useful line magics include:

```
1 %pwd
2 %cd path/to/directory
3 %run script.py
4 %who
5 %whos
6 %time expression
7 %timeit expression
```

The `%pwd` command displays the current working directory:

```
1 %pwd
```

The `%cd` command changes the current working directory:

```
1 %cd path/to/project
```

A Python script can be executed from Jupyter using `%run`:

```
1 %run script.py
```

IPython also distinguishes between **magic commands** and **shell commands**. A command prefixed with `%` is interpreted by IPython, whereas a command prefixed with `!` is passed to the operating-system shell:

```
1 %pwd
2 !python --version
```

1.3 Python Modules: Reusable Code and Namespaces

A **module** is a Python file containing definitions and statements that can be reused by other parts of a program. Modules provide a fundamental mechanism for organizing related functionality into separate, reusable units.

A module may contain functions, classes, variables, constants, and executable statements. Its definitions can be made available to another module or script using the `import` statement:

```
1 import math
2
3 radius = 5
4 area = math.pi * radius**2
5
6 area
```

78.53981633974483

Modules provide several practical advantages:

- **Code organization** — related functionality can be grouped into logical units, simplifying the structure, comprehension, and maintenance of a codebase.
- **Reusability** — functionality defined in a module can be reused across different parts of a program or across separate projects, reducing code duplication and promoting consistency.
- **Namespace management** — each module defines its own namespace, helping to prevent naming conflicts and providing context for variables, functions, and classes.
- **Encapsulation** — modules provide abstraction boundaries that separate implementation details from the interfaces used by other parts of a program.

1.3.1 Importing Modules

A module can be imported directly:

```
1 import math
2
3 math.sqrt(16)
```

4.0

An alias can be assigned to a module when it is imported:

```
1 import math as mt
2
3 mt.sqrt(16)
```

4.0

Aliases are particularly common for frequently used Data Science libraries:

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
```

Specific objects can also be imported directly from a module:

```
1 from math import sqrt, pi
2
3 sqrt(16)
```

4.0

Imported objects can themselves be assigned aliases:

```
1 from math import factorial as fact
2
3 fact(5)
```

120

Python also supports wildcard imports:

```
1 from module_name import *
```

This form imports all public names from a module into the current namespace. In general, explicit imports are preferable because they make dependencies clearer and reduce the possibility of naming conflicts.

1.3.2 Inspecting Modules and Namespaces

A **namespace** is a mapping between names and Python objects. Modules define their own namespaces, allowing identifiers to be organized without necessarily introducing them into the namespace of the importing program.

The built-in `dir()` function can be used to inspect the names associated with an object:

```
1 import math
2
3 dir(math)
```

```
['__doc__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 'acos',
 'acosh',
 'asin',
 'asinh',
 'atan',
 'atan2',
 'atanh',
 'cbrt',
 'ceil',
 'comb',
 'copysign',
 'cos',
 'cosh',
 'degrees',
 'dist',
 'e',
 'erf',
 'erfc',
 'exp',
 'exp2',
 'expm1',
 'fabs',
 'factorial',
 'floor',
 'fma',
 'fmod',
 'frexp',
 'fsum',
 'gamma',
 'gcd',
 'hypot',
 'inf',
 'isclose',
 'isfinite',
 'isinf',
 'isnan',
 'isqrt',
 'lcm',
 'ldexp',
```

```
'lgamma',  
'log',  
'log10',  
'log1p',  
'log2',  
'modf',  
'nan',  
'nextafter',  
'perm',  
'pi',  
'pow',  
'prod',  
'radians',  
'remainder',  
'sin',  
'sinh',  
'sqrt',  
'sumprod',  
'tan',  
'tanh',  
'tau',  
'trunc',  
'ulp']
```

When called without an argument:

```
1 dir()
```

`dir()` provides the names available in the current scope.

For many imported modules, the `__file__` attribute identifies the file from which the module was loaded:

```
1 import numpy as np  
2  
3 np.__file__
```

Not every module necessarily provides a `__file__` attribute. Built-in modules, for example, may not correspond to a conventional source file.

Python also provides the built-in `help()` system for inspecting documentation:

```
1 help(math)  
2 help(math.sqrt)
```

Within IPython and Jupyter, documentation can also be inspected interactively using `?`:

```
1 math.sqrt?
```

and, when additional implementation information is available:

```
1 math.sqrt??
```

1.3.3 Module Loading and Reloading

When a module is imported, Python normally caches the resulting module object in `sys.modules`. Importing the same module again within the same Python process therefore does not normally execute its top-level code again.

This behaviour is particularly relevant during interactive development in Jupyter. If the source code of an imported module changes, the object already loaded in memory may continue to represent its previous state.

Restarting the kernel reconstructs the Python process from a clean state. Alternatively, a module can be reloaded explicitly using `importlib`:

```
1 import importlib
2 import my_module
3
4 importlib.reload(my_module)
```

IPython also provides the autoreload extension:

```
1 %load_ext autoreload
2 %autoreload 2
```

This can be useful during interactive development because imported modules can be automatically reloaded when their source code changes.

1.4 Python Execution from the Command Line

Python can be started interactively from a terminal:

```
1 python
```

or, depending on the system:

```
1 python3
```

The interactive interpreter provides direct access to Python's built-in documentation system:

```
1 help()
```

Documentation for a specific object can be requested directly:

```
1 help(str)
2 help(print)
```

The interactive interpreter can be exited with:

```
1 exit()
```

or:

```
1 quit()
```

On Unix-like operating systems, a Python script can specify its interpreter using a **shebang** at the beginning of the file:

```
1 #!/usr/bin/env python3
```

The command:

```
1 chmod +x script.py
```

adds executable permission to the script. It can then be executed directly:

```
1 ./script.py
```

The shebang and executable permission mechanism is primarily relevant to Unix-like operating systems such as Linux and macOS. On Windows, Python scripts are normally executed through the Python interpreter, the Python launcher, or a development environment.

1.5 Environment and Package Management with Conda

A **Python environment** provides an isolated context containing a particular Python interpreter together with a defined collection of installed packages and dependencies.

Isolated environments are useful because different projects may require different Python versions or different versions of the same libraries. Environment management reduces dependency conflicts and helps make computational environments reproducible.

Conda provides both environment management and package management.

Available Conda environments can be listed with:

```
1 conda env list
```

or:

```
1 conda info --envs
```

A new environment can be created with a specific Python version:

```
1 conda create --name my_env python=3.12
```

An environment can be activated with:

```
1 conda activate my_env
```

and deactivated with:

```
1 conda deactivate
```

Packages installed in the active environment can be listed with:

```
1 conda list
```

Packages in a specific environment can be inspected without activating it:

```
1 conda list --name my_env
```

A package can be installed into the active environment:

```
1 conda install numpy
```

or directly into a specified environment:

```
1 conda install --name my_env numpy
```

An environment and all of its packages can be removed with:

```
1 conda remove --name my_env --all
```

Conda itself can be updated with:

```
1 conda update conda
```

All packages in a particular environment can be updated with:

```
1 conda update --name my_env --all
```

For reproducibility, the specification of an environment can be exported to a YAML file:

```
1 conda env export --name my_env > environment.yml
```

An environment can subsequently be recreated from that specification:

```
1 conda env create --file environment.yml
```

The resulting environment.yml provides a portable description of the environment and its dependencies.

1.5.1 Reference Resources

Official documentation and project resources:

- [Python](#) — Official Python website, language documentation, and downloads.
- [Python Standard Library](#) — Reference documentation for modules included with Python.
- [IPython](#) — Documentation for IPython, including interactive features and magic commands.
- [Jupyter](#) — Documentation and resources for the Jupyter ecosystem.
- [Conda](#) — Documentation for Conda package and environment management.
- [Anaconda](#) — Python distribution and platform commonly used for scientific computing and Data Science.

1.6 The Python Ecosystem

Python's open-source nature has supported the development of a broad ecosystem of specialized libraries, particularly for scientific computing, Data Science, and Machine Learning.

Some of the most widely used libraries and modules include:

- [pandas](#) — Data manipulation and analysis library providing data structures such as the DataFrame for working with structured and tabular data.
- [NumPy](#) — Core numerical computing library providing multidimensional arrays and a broad collection of mathematical operations.
- [Matplotlib](#) — Visualization library for creating static, animated, and interactive plots and figures.
- [math](#) — Standard-library module providing mathematical functions for real-valued calculations.
- [cmath](#) — Standard-library module providing mathematical functions for complex-valued calculations.
- [SciPy](#) — Scientific computing library built on NumPy, providing functionality for optimization, integration, interpolation, signal processing, statistics, linear algebra, and related numerical methods.
- [scikit-learn](#) — Machine Learning library providing consistent interfaces for tasks such as classification, regression, clustering, dimensionality reduction, preprocessing, model selection, and evaluation.
- [TensorFlow](#) — Machine Learning framework for numerical computation and the development and deployment of Machine Learning and deep learning models.
- [PyTorch](#) — Machine Learning framework widely used for deep learning, research, and model development.
- [Keras](#) — High-level API for building and training neural networks.

Together with Python's standard library, these external libraries provide a broad computational environment for tasks ranging from general-purpose programming and numerical computation to data analysis, visualization, Machine Learning, and scientific computing.

1.7 Representing Information in Python

A fundamental question in any programming language is what kinds of information the language can represent and manipulate. This is directly related to the computational process introduced earlier: input → processing → output. Information must first be represented in a form that a program can operate on before any meaningful computation can take place.

In Python, every value is an **object**, and every object has a **type**. A type determines the nature of the value, the operations that can be performed on it, and aspects of how it behaves during program execution.

Python provides several built-in types for representing individual values, including:

- `bool` — Boolean values used to represent logical states.
- `int` — integer numbers.
- `float` — floating-point numbers.
- `complex` — complex numbers.
- `str` — sequences of Unicode characters used to represent text.

Python also provides built-in data structures for organizing multiple objects, including:

- `list` — ordered and mutable collections.
- `tuple` — ordered and immutable collections.
- `dict` — mappings between keys and values.
- `set` — unordered collections of unique elements.

The broader Python ecosystem provides additional data structures designed for numerical and data-oriented computation. In particular, NumPy provides multidimensional arrays through `ndarray`, while pandas provides structures such as `Series` and `DataFrame` for working with labelled and tabular data.

Understanding these types and structures requires understanding how Python associates objects with names. This introduces the concept of **variables**.

2 Working with Python Variables

In Python, a variable is best understood as a **name bound to an object**. Assignment creates an association between a name and an object, allowing that object to be referenced and manipulated throughout a program.

Variables do not need to be declared in advance. A name is created when an object is assigned to it using the assignment operator `=`:

```
1 x = 1
2 x
```

1

Another name can refer to an object of a different type:

```
1 message = "hello"
2 message
```

'hello'

The = symbol represents **assignment**, not mathematical equality. The expression on the right-hand side is evaluated first, and the resulting object is then associated with the name on the left-hand side.

For example:

```
1 x = 3
2 y = x + 1
3
4 y
```

4

Here, $x + 1$ is evaluated first, producing the integer object 4, which is then assigned to *y*.

2.1 Multiple Assignment

Python allows several names to be assigned in a single statement:

```
1 x, y, z = 10, 20, 30
2
3 print(x)
4 print(y)
5 print(z)
```

10
20
30

This syntax is based on **unpacking**: the values on the right-hand side are assigned to the corresponding names on the left-hand side.

It is particularly useful when working with related values or when unpacking elements from data structures.

For example, two variables can be swapped without requiring an explicit temporary variable:

```
1 x = 10
2 y = 20
3
4 x, y = y, x
5
6 print(x)
7 print(y)
```

```
20
10
```

2.2 Variable Naming

Variable names are identifiers and must follow Python’s identifier rules. In general, an identifier:

- may contain letters, digits, and underscores;
- cannot begin with a digit;
- cannot contain spaces;
- cannot contain characters such as @, #, \$, or %;
- cannot be a reserved Python keyword.

Python identifiers are **case-sensitive**, so the following names refer to different variables:

```
1 age
2 Age
3 AGE
```

Python supports Unicode characters in identifiers. In practice, English-language identifiers are generally preferable for code intended to be shared across teams and environments.

Beyond the syntactic rules, consistent naming conventions improve readability. Python code conventionally follows the recommendations defined in **PEP 8**.

Common naming styles include:

- snake_case — lowercase words separated by underscores; the standard convention for variables and functions in Python, e.g. `user_age`, `total_count`, `average_height`.
- PascalCase — words begin with capital letters; conventionally used for class names, e.g. `User`, `CustomerProfile`, `DataProcessor`.

- camelCase — the first word begins with lowercase and subsequent words are capitalized, e.g. `userAge`, `totalCount`. This style is common in other languages but is not the standard convention for Python variables and functions.
- kebab-case — words separated by hyphens, e.g. `user-age`. This form is used in contexts such as URLs and configuration files but is **not valid as a Python identifier**, because `-` is interpreted as the subtraction operator.

Meaningful names should describe the role of the object they reference. For example:

```
1 average_height = 1.76
2 customer_count = 125
3 model_accuracy = 0.91
```

is generally clearer than:

```
1 a = 1.76
2 b = 125
3 c = 0.91
```

Short names remain appropriate when their meaning is conventional or immediately clear from the surrounding context, such as `i` in a short loop or `x` and `y` in mathematical expressions.

2.3 Python Keywords

Python reserves a set of words with predefined syntactic meanings. These words cannot be used as variable names.

The current list can be inspected using the `keyword` module:

```
1 import keyword
2
3 keyword.kwlist
```

```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
```

```
'class',  
'continue',  
'def',  
'del',  
'elif',  
'else',  
'except',  
'finally',  
'for',  
'from',  
'global',  
'if',  
'import',  
'in',  
'is',  
'lambda',  
'nonlocal',  
'not',  
'or',  
'pass',  
'raise',  
'return',  
'try',  
'while',  
'with',  
'yield']
```

It is generally preferable to inspect the list programmatically rather than maintain a static list, since the language can evolve between Python versions.

2.4 Reassignment and Operations with Variables

Once a name has been assigned to a numerical object, it can be used in expressions:

```
1 x = 3  
2  
3 x + 1
```

4

The result of an expression can be assigned to another name:

```
1 y = x + 1  
2  
3 y
```

4

A name can also be **reassigned**:

```
1 x = 3
2 x = x + 1
3
4 x
```

4

This statement is valid because assignment is not mathematical equality. Python first evaluates the expression `x + 1` using the object currently associated with `x`, and then rebinds `x` to the resulting object.

Python provides **augmented assignment operators** as a concise form for common update operations:

```
1 x = 7
2 x += 2
3
4 x
```

9

The most common augmented assignment operators include:

Operator	Example	Corresponding operation
<code>+=</code>	<code>x += y</code>	Addition
<code>-=</code>	<code>x -= y</code>	Subtraction
<code>*=</code>	<code>x *= y</code>	Multiplication
<code>/=</code>	<code>x /= y</code>	Division
<code>//=</code>	<code>x //= y</code>	Floor division
<code>%=</code>	<code>x %= y</code>	Modulo
<code>**=</code>	<code>x **= y</code>	Exponentiation

Table 1: Common augmented assignment operators in Python.

Although expressions such as `x += y` are often informally described as shorthand for `x = x + y`, their behaviour can differ for some mutable objects because augmented assignment may modify an object in place. This distinction becomes relevant when considering mutability and object references.

2.5 Comments

Comments provide contextual information within source code without being executed by the Python interpreter. A single-line comment begins with #:

```
1 # Calculate the area of a circle
2 radius = 5
3 area = math.pi * radius**2
```

Comments can also appear after a statement:

```
1 max_iterations = 100 # Upper limit for the optimisation procedure
```

Comments are most useful when they explain **why** a decision was made, clarify an assumption, document a non-obvious constraint, or provide context that cannot be expressed clearly through the code itself.

Comments that merely restate obvious code generally add noise rather than useful information. For example:

```
1 # Assign 10 to x
2 x = 10
```

provides little additional information.

In general, clear naming and straightforward program structure should make the behaviour of the code understandable, while comments should provide context where the code alone cannot adequately communicate intent.

3 Problems

The following problems are intended to reinforce the concepts developed throughout these notes. They focus on conceptual understanding, interpretation of Python behaviour, and technical reasoning rather than on writing substantial programs.

3.1 Problem 1 — From Algorithm to Execution

Consider the following statements:

1. An algorithm is the same thing as a Python program.
2. A high-level programming language provides abstractions that reduce the need to work directly with machine-level operations.
3. Python source code is executed directly by the processor without any intermediate representation.
4. A programming language provides a formal way to express algorithms in a form that can ultimately be executed by a computer.

For each statement:

1. Determine whether it is correct.
2. If it is incorrect, identify the conceptual error.
3. Explain the relationship among an **algorithm**, a **programming language**, a **program**, and the underlying computer hardware.

3.2 Problem 2 — Compiled or Interpreted?

Python is often described as an interpreted language.

3.2.1 Questions

1. Why is this description useful but incomplete?
2. Describe, at a high level, what happens to Python source code when it is executed by CPython.
3. What is Python bytecode?
4. What role does the Python virtual machine play?
5. Why should the distinction between compiled and interpreted languages not be treated as absolute?

3.3 Problem 3 — Programming Paradigms

Python supports procedural, object-oriented, and functional programming styles.

3.3.1 Questions

1. What does it mean to describe Python as a **multi-paradigm language**?
2. Does using one paradigm prevent a program from using another?
3. Why might different paradigms be useful for different parts of the same application?
4. Give a conceptual example of a task that could naturally be expressed procedurally and another that could naturally be represented through objects.

3.4 Problem 4 — Script or Interactive Environment?

Consider the following tasks:

1. Exploring a newly received dataset.
2. Developing a reusable command-line program.
3. Testing a mathematical expression quickly.
4. Creating a reproducible production pipeline.
5. Experimenting interactively with plots and intermediate calculations.

3.4.1 Questions

1. Which tasks are naturally suited to an interactive environment such as Jupyter?
2. Which are more naturally suited to Python scripts or modules?
3. What advantages does interactive computing provide?
4. What risks or limitations can arise when a complex workflow depends heavily on notebook execution state?
5. Why can the distinction matter for reproducibility?

3.5 Problem 5 — Magic Commands and Shell Commands

Consider the following commands used inside an IPython or Jupyter session:

```
1 %pwd
2 %run analysis.py
3 !python --version
4 %timeit expression
```

3.5.1 Questions

1. Which commands are interpreted by IPython?
2. Which command is passed to the operating-system shell?
3. What is the difference between `%run analysis.py` and executing `python analysis.py` from a terminal?
4. Why are magic commands not part of the Python language itself?
5. Would a `.py` script containing `%pwd` necessarily execute successfully with the standard Python interpreter? Explain.

3.6 Problem 6 — Modules and Namespaces

Suppose a module named `statistics_tools.py` contains:

```
1 threshold = 0.8
2
3 def score(x):
4     return x * 2
```

Another program executes:

```
1 import statistics_tools
```

3.6.1 Questions

1. What namespace is created or accessed by the module?
2. How would the program refer to `threshold`?
3. How would it call `score()`?
4. Why does `import statistics_tools` reduce the risk of naming collisions compared with importing every name directly?
5. What conceptual role does the module name play in expressions such as `statistics_tools.score(...)`?

3.7 Problem 7 — Import Styles

Consider the following alternatives:

```
1 import math
```

```
1 import math as mt
```

```
1 from math import sqrt, pi
```

```
1 from math import *
```

3.7.1 Questions

1. How does each form affect the current namespace?
2. Which forms preserve the module name as part of the reference?
3. What advantages can aliases provide?
4. Why are explicit imports generally preferable to wildcard imports?
5. Under what circumstances could two wildcard imports create a subtle error?
6. Which import style would you generally prefer in shared production code, and why?

3.8 Problem 8 — Inspecting a Module

Suppose you import a module:

```
1 import math
```

3.8.1 Questions

1. What information does `dir(math)` provide?
2. How is `dir()` without an argument conceptually different?
3. What is the purpose of `help(math.sqrt)`?
4. In IPython, what is the purpose of `math.sqrt??`?
5. What additional information may `math.sqrt??` provide?
6. Why might `module.__file__` not exist for every module?

3.9 Problem 9 — Module Caching

Suppose a module contains:

```
1 print("module loaded")
```

and is imported twice during the same Python process:

```
1 import my_module
2 import my_module
```

3.9.1 Questions

1. Would you normally expect "module loaded" to be printed once or twice?
2. Why?
3. What role does `sys.modules` play?
4. Suppose the source file changes after the first import. Why might importing it again not reflect the change?
5. What are three ways to obtain the updated module during interactive development?
6. What is the conceptual difference between restarting the kernel and reloading only one module?

3.10 Problem 10 — Python Environments

Two projects require different package versions:

- Project A requires Python 3.11 and version 1 of a library.
- Project B requires Python 3.12 and version 2 of the same library.

3.10.1 Questions

1. Why can installing all dependencies globally create problems?
2. How does an isolated environment help?
3. What does a Python environment contain conceptually?
4. Why is environment isolation important for reproducibility?
5. What information does an `environment.yml` file attempt to preserve?
6. Does recreating an environment guarantee that all external aspects of an analysis are reproducible? Explain.

3.11 Problem 11 — Conda Commands

Explain the purpose of each command:

```
1 conda env list
```

```
1 conda create --name ds_env python=3.12
```

```
1 conda activate ds_env
```

```
1 conda install numpy
```

```
1 conda list
```

```
1 conda env export --name ds_env > environment.yml
```

```
1 conda env create --file environment.yml
```

3.11.1 Follow-up Questions

1. What is the difference between creating an environment and activating it?
2. What happens conceptually when a package is installed while an environment is active?
3. Why might an environment specification be committed to version control?

3.12 Problem 12 — Python Ecosystem

Consider the following tasks:

1. Numerical array computation.
2. Tabular data manipulation.
3. Static plotting.
4. Scientific optimisation and integration.
5. Conventional Machine Learning.
6. Deep-learning model development.

3.12.1 Questions

1. Which libraries mentioned in these notes are commonly associated with each task?
2. Why is NumPy foundational to much of the scientific Python ecosystem?
3. How does pandas differ conceptually from NumPy?
4. What role does scikit-learn typically occupy relative to TensorFlow or PyTorch?
5. Why should familiarity with libraries not be confused with proficiency in Python itself?

3.13 Problem 13 — Objects and Types

Consider the following objects:

```
1 True
2 42
3 3.14
4 2 + 3j
5 "data"
```

3.13.1 Questions

1. Identify the built-in type of each object.
2. What does it mean to say that every Python value is an object?
3. What role does an object's type play?
4. Can two objects contain apparently similar information while having different types?
5. Why does type matter when determining which operations are valid?

3.14 Problem 14 — Scalar Types and Data Structures

Classify the following Python types according to whether they normally represent individual values or collections of values:

```
1 bool
2 int
3 float
4 complex
5 str
6 list
7 tuple
8 dict
9 set
```

3.14.1 Questions

1. Which collection types are ordered?
2. Which are mutable?
3. Which are mappings?
4. Which enforce uniqueness of elements?

5. Why is str somewhat unusual when distinguishing scalar values from collections or sequences?

3.15 Problem 15 — What Is a Variable in Python?

Consider:

```
1 x = 10
```

3.15.1 Questions

1. What exactly is x?
2. What exactly is 10?
3. Does Python store the integer value *inside* the variable in the same conceptual sense as a labelled box?
4. What does it mean to say that x is **bound to an object**?
5. Why is this description more useful when later studying mutability and references?

3.16 Problem 16 — Assignment Is Not Equality

Consider:

```
1 x = 3
2 x = x + 1
```

3.16.1 Questions

1. Why is the second statement valid in Python even though the corresponding mathematical equation

$$x = x + 1$$

would be inconsistent over ordinary numbers? 2. Describe the order in which Python evaluates the second statement. 3. What object is associated with x before the second assignment? 4. What object is associated with x afterwards? 5. Why is the term **reassignment** appropriate?

3.17 Problem 17 — Names and Objects

Consider:

```
1 x = 10
2 y = x
```

3.17.1 Questions

1. What does $y = x$ conceptually do?
2. Does it necessarily create a completely independent copy of the object associated with x ?
3. How many names are involved?
4. How many integer objects need to be involved conceptually?
5. Why will this distinction become important when mutable objects are introduced?

3.18 Problem 18 — Multiple Assignment

Consider:

```
1 x, y, z = 10, 20, 30
```

3.18.1 Questions

1. Explain this statement in terms of unpacking.
2. Which value is associated with each name?
3. What condition must hold between the number of values and the number of receiving names in this simple case?
4. Predict conceptually what would happen if there were three names but only two values.

3.19 Problem 19 — Variable Swapping

Consider:

```
1 x = 10
2 y = 20
3
4 x, y = y, x
```


3.19.1 Questions

1. What are the final values associated with x and y?
2. Why is an explicit temporary variable unnecessary?
3. Explain the operation using the idea of evaluating the right-hand side before performing assignment.
4. How would the equivalent swap normally be written in a language without this unpacking syntax?

3.20 Problem 20 — Valid and Invalid Identifiers

Determine which of the following are valid Python identifiers:

```
1 customer_age
2 2nd_customer
3 customer2
4 customer-age
5 _customer
6 customer age
7 total$
8 Age
9 age
10 class
11 model_accuracy
```

For every invalid identifier, explain why it is invalid.

3.20.1 Follow-up Questions

1. Are Age and age the same identifier?
2. Can Python identifiers contain Unicode characters?
3. Why might Unicode identifiers nevertheless be undesirable in shared technical code?

3.21 Problem 21 — Naming Conventions

Consider the following names:

```
1 customer_age
2 CustomerAge
3 customerAge
4 customer-age
5 x
6 avg_h
7 average_customer_height
```

3.21.1 Questions

1. Identify the naming style represented by each.
2. Which style is conventionally used for Python variables and functions?
3. Which style is conventionally used for class names?
4. Which example is not a valid Python identifier?
5. Is a one-letter name such as `x` always poor style?
6. When are short mathematical names appropriate?
7. Why is `average_customer_height` generally more informative than `a` in application code?

3.22 Problem 22 — Python Keywords

Suppose a programmer writes:

```
1 class = "A"
```

3.22.1 Questions

1. Why is this invalid?
2. What is a Python keyword?
3. How can the current keyword list be inspected programmatically?
4. Why is inspecting the keyword list preferable to maintaining a manually written list in long-lived notes or software?

3.23 Problem 23 — Reassignment and Dynamic Typing

Consider:

```
1 x = 10
2 x = "ten"
```

3.23.1 Questions

1. Is this valid Python?
2. Has the name `x` changed type?
3. Or is it more precise to say that `x` has been rebound to an object of a different type?
4. What does this example suggest about Python's variable model?
5. Why can imprecise language such as "the variable stores a type" become confusing?

3.24 Problem 24 — Augmented Assignment

Consider:

```
1 x = 7
2 x += 2
```

3.24.1 Questions

1. What is the final value associated with `x`?
2. Why is `x += 2` often described informally as equivalent to `x = x + 2`?
3. Why is that equivalence not universally exact for all Python objects?
4. What role does mutability play in the difference?
5. Why is this distinction largely invisible when using immutable integers?

3.25 Problem 25 — Predict the Result of Augmented Assignment

Without executing the code, determine the final value of `x` in each case:

```
1 x = 20
2 x -= 3
```

```
1 x = 5
2 x *= 4
```

```
1 x = 17
2 x //= 5
```

```
1 x = 17
2 x %= 5
```

```
1 x = 2
2 x **= 5
```

3.25.1 Follow-up Question

Explain the conceptual difference among ordinary division, floor division, modulo, and exponentiation.

3.26 Problem 26 — Comments and Code Quality

Consider the following two fragments:

```
1 # Assign 100 to max_iterations
2 max_iterations = 100
```

and

```
1 # Limit iterations because the optimisation procedure may otherwise
2 # continue for an impractically long time on poorly scaled data.
3 max_iterations = 100
```

3.26.1 Questions

1. Which comment is more useful?
2. Why?
3. What kinds of information should comments ideally communicate?
4. Why can comments that merely restate code reduce readability?
5. What should be preferred when meaning can be communicated through clear naming alone?

3.27 Problem 27 — Namespace Collision

Suppose two modules contain functions with the same name:

```
1 module_a.transform
2 module_b.transform
```

A programmer writes:

```
1 from module_a import transform
2 from module_b import transform
```

3.27.1 Questions

1. Which transform name will be available after both imports?
2. What problem has occurred conceptually?
3. How could module-qualified imports avoid the ambiguity?
4. Why is this an argument against indiscriminate wildcard imports?

3.28 Problem 28 — Reproducibility in Interactive Computing

A data scientist performs the following actions in a notebook:

1. Imports a module.
2. Modifies the module's source file externally.
3. Runs a later notebook cell expecting the new behaviour.
4. Receives results corresponding to the old implementation.

3.28.1 Questions

1. Explain why this can happen.
2. How could `importlib.reload()` help?
3. How could IPython's autoreload extension help?
4. Why might restarting the kernel still be the safest option in some situations?
5. What broader lesson does this illustrate about hidden execution state in interactive environments?

3.29 Problem 29 — Environment Reproducibility Scenario

A colleague sends you a Python project together with an `environment.yml` file.

The project worked correctly on their computer but fails on yours.

3.29.1 Questions

1. What problem is the environment file intended to solve?
2. What command could be used to recreate the environment?
3. Name several reasons why the project could still fail even after recreating the environment successfully.
4. Why does dependency reproducibility not necessarily imply complete computational reproducibility?

3.30 Problem 30 — Integrated Technical Scenario

A data scientist creates a Conda environment, starts JupyterLab, imports a custom module, loads pandas and NumPy using conventional aliases, and then creates several variables during an analysis.

Later, the custom module is edited outside Jupyter, a variable is rebound to an object of another type, and the notebook is executed out of order.

3.30.1 Questions

Explain the concepts involved in this scenario, including:

1. environment isolation;
2. interactive computing;
3. module imports;
4. module caching;
5. namespaces;
6. aliases;
7. variables as names bound to objects;
8. reassignment;
9. object types;
10. execution state.

Finally, explain why understanding these foundational concepts is important even when most day-to-day Data Science work is performed through high-level libraries such as pandas, NumPy, or scikit-learn.